

Chapter 9

SQL in a Server Environment

We now turn to the question of how SQL fits into a complete programming environment. The typical server environment is introduced in Section 9.1. Section 9.2 introduces the SQL terminology for client-server computing and connecting to a database.

Then, we turn to how programming is really done, when SQL must be used to access a database as part of a typical application. In Section 9.3 we see how to embed SQL in programs that are written in an ordinary programming language, such as C. A critical issue is how we move data between SQL relations and the variables of the surrounding, or “host,” language. Section 9.4 considers another way to combine SQL with general-purpose programming: persistent stored modules, which are pieces of code stored as part of a database schema and executable on command from the user.

A third programming approach is a “call-level interface,” where we program in some conventional language and use a library of functions to access the database. In Section 9.5 we discuss the SQL-standard library called SQL/CLI, for making calls from C programs. Then, in Section 9.6 we meet Java’s JDBC (database connectivity), which is an alternative call-level interface. Finally, another popular call-level interface, PHP, is covered in Section 9.7.

9.1 The Three-Tier Architecture

Databases are used in many different settings, including small, standalone databases. For example, a scientist may run a copy of MySQL or Microsoft Access on a laboratory computer to store experimental data. However, there is a very common architecture for large database installations; this architecture motivates the discussion of the entire chapter. The architecture is called *three-tier* or *three-layer*, because it distinguishes three different, interacting functions:

1. *Web Servers.* These are processes that connect clients to the database system, usually over the Internet or possibly a local connection.
2. *Application Servers.* These processes perform the “business logic,” whatever it is the system is intended to do.
3. *Database Servers.* These processes run the DBMS and perform queries and modifications at the request of the application servers.

The processes may all run on the same processor in a small system, but it is common to dedicate a large number of processors to each of the tiers. Figure 9.1 suggests how a large database installation would be organized.

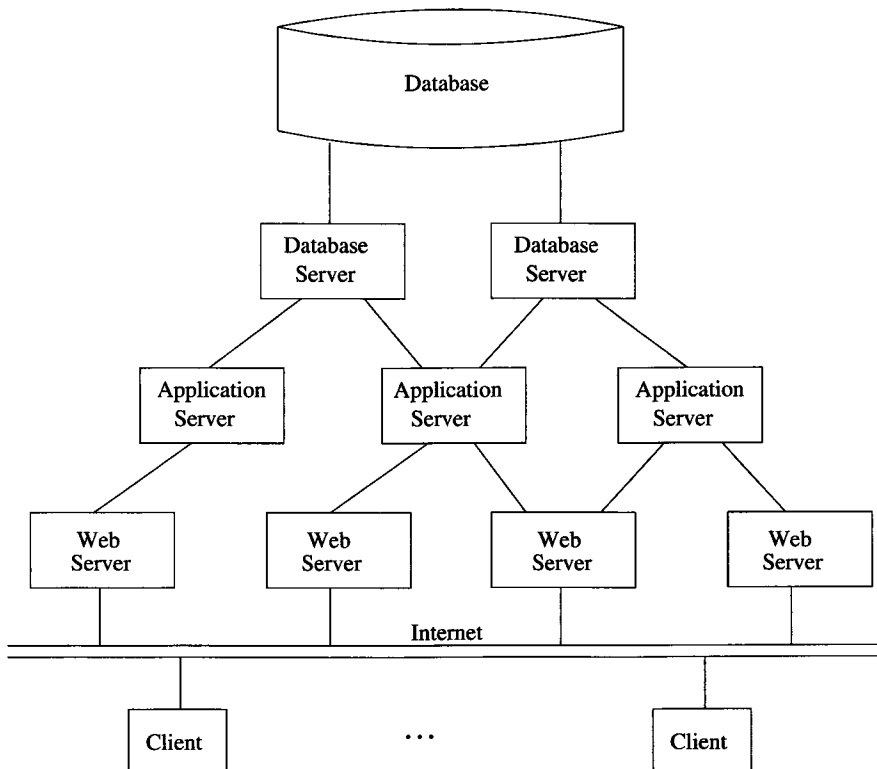


Figure 9.1: The Three-Tier Architecture

9.1.1 The Web-Server Tier

The web-server processes manage the interactions with the user. When a user makes contact, perhaps by opening a URL, a web server, typically running

Apache/Tomcat, responds to the request. The user then becomes a *client* of this web-server process. Typically, the client's actions are performed by the web-browser, e.g., managing of the filling of forms, which are then posted to the web server.

As an example, let us consider a site such as Amazon.com. A user (customer) opens a connection to the Amazon database system by entering the URL `www.amazon.com` into their browser. The Amazon web-server presents a "home page" to the user, which includes forms, menus, and buttons enabling the user to express what it is they want to do. For example, the user may set a menu to Books and enter into a form the title of the book they are interested in. The client web-browser transmits this information to the Amazon web-server, and that web-server must negotiate with the next tier — the application tier — to fulfill the client's request.

9.1.2 The Application Tier

The job of the application tier is to turn data, from the database, into a response to the request that it receives from the web-server. Each web-server process can invoke one or more application-tier processes to handle the request; these processes can be on one machine or many, and they may be on the same or different machines from the web-server processes.

The actions performed by the application tier are often referred to as the *business logic* of the organization operating the database. That is, one designs the application tier by reasoning out what the response to a request by the potential customer should be, and then implementing that strategy.

In the case of our example of a book at Amazon.com, this response would be the elements of the page that Amazon displays about a book. That data includes the title, author, price, and several other pieces of information about the book. It also includes links to more information, such as reviews, alternative sellers of the book, and similar books.

In a simple system, the application tier may issue database queries directly to the database tier, and assemble the results of those queries, perhaps in an HTML page. In a more complex system, there can be several subtiers to the application tier, and each may have its own processes. A common architecture is to have a subtier that supports "objects." These objects can contain data such as the title and price of a book in a "book object." Data for this object is obtained by a database query. The object may also have methods that can be invoked by the application-tier processes, and these methods may in turn cause additional queries to be issued to the database when and if they are invoked.

Another subtier may be present to support *database integration*. That is, there may be several quite independent databases that support operations, and it may not be possible to issue queries involving data from more than one database at a time. The results of queries to different sources may need to be combined at the integration subtier. To make integration more complex, the databases may not be compatible in a number of important ways. We shall

examine the technology of information integration elsewhere. However, for the moment, consider the following hypothetical example.

Example 9.1: The Amazon database containing information about a book may have a price in dollars. But the customer is in Europe, and their account information is in another database, located in Europe, with billing information in Euros. The integration subtier needs to know that there is a difference in currencies, when it gets a price from the books database and uses that price to enter data into a bill that is displayed to the customer. □

9.1.3 The Database Tier

Like the other tiers, there can be many processes in the database tier, and the processes can be distributed over many machines, or all be together on one. The database tier executes queries that are requested from the application tier, and may also provide some buffering of data. For example, a query that produces many tuples may be fed one-at-a-time to the requesting process of the application tier.

Since creating connections to the database takes significant time, we normally keep a large number of connections open and allow application processes to share these connections. Each application process must return the connection to the state in which it was found, to avoid unexpected interactions between application processes.

The balance of this chapter is about how we implement a database tier. Especially, we need to learn:

1. How do we enable a database to interact with “ordinary” programs that are written in a conventional language such as C or Java?
2. How do we deal with the differences in data-types supported by SQL and conventional languages? In particular, relations are the results of queries, and these are not directly supported by conventional languages.
3. How do we manage connections to a database when these connections are shared between many short-lived processes?

9.2 The SQL Environment

In this section we shall take the broadest possible view of a DBMS and the databases and programs it supports. We shall see how databases are defined and organized into clusters, catalogs, and schemas. We shall also see how programs are linked with the data they need to manipulate. Many of the details depend on the particular implementation, so we shall concentrate on the general ideas that are contained in the SQL standard. Sections 9.5, 9.6, and 9.7 illustrate how these high-level concepts appear in a “call-level interface,” which requires the programmer to make explicit connections to databases.

9.2.1 Environments

A *SQL environment* is the framework under which data may exist and SQL operations on data may be executed. In practice, we should think of a SQL environment as a DBMS running at some installation. For example, ABC company buys a license for the Megatron 2010 DBMS to run on a collection of ABC's machines. The system running on these machines constitutes a SQL environment.

All the database elements we have discussed — tables, views, triggers, and so on — are defined within a SQL environment. These elements are organized into a hierarchy of structures, each of which plays a distinct role in the organization. The structures defined by the SQL standard are indicated in Fig. 9.2.

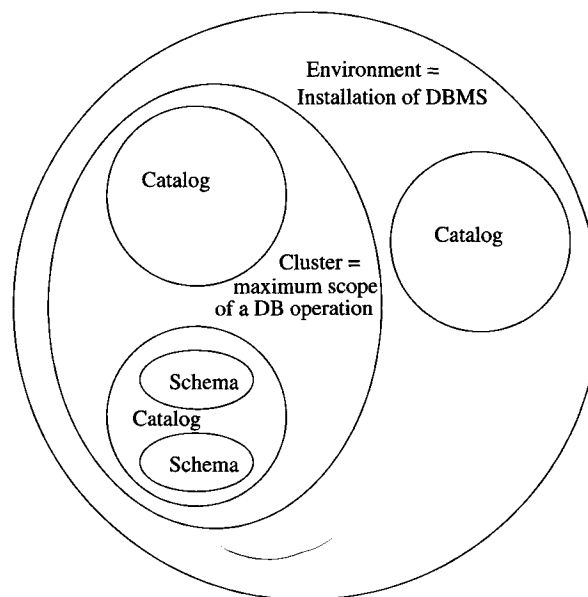


Figure 9.2: Organization of database elements within the environment

Briefly, the organization consists of the following structures:

1. *Schemas*. These are collections of tables, views, assertions, triggers, and some other types of information (see the box on “More Schema Elements” in Section 9.2.2). Schemas are the basic units of organization, close to what we might think of as a “database,” but in fact somewhat less than a database as we shall see in point (3) below.
2. *Catalogs*. These are collections of schemas. They are the basic unit for supporting unique, accessible terminology. Each catalog has one or more schemas; the names of schemas within a catalog must be unique, and

each catalog contains a special schema called `INFORMATION_SCHEMA` that contains information about all the schemas in the catalog.

3. *Clusters.* These are collections of catalogs. Each user has an associated cluster: the set of all catalogs accessible to the user (see Section 10.1 for an explanation of how access to catalogs and other elements is controlled). A cluster is the maximum scope over which a query can be issued, so in a sense, a cluster is “the database” as seen by a particular user.

9.2.2 Schemas

The simplest form of schema declaration is:

```
CREATE SCHEMA <schema name> <element declarations>
```

The element declarations are of the forms discussed in various places, such as Sections 2.3, 8.1.1, 7.5.1, and 9.4.1.

Example 9.2: We could declare a schema that includes the five relations about movies that we have been using in our running example, plus some of the other elements we have introduced, such as views. Figure 9.3 sketches the form of such a declaration. □

```
CREATE SCHEMA MovieSchema
  CREATE TABLE MovieStar ... as in Fig. 7.3
    Create-table statements for the four other tables
  CREATE VIEW MovieProd ... as in Example 8.2
    Other view declarations
  CREATE ASSERTION RichPres ... as in Example 7.11
```

Figure 9.3: Declaring a schema

It is not necessary to declare the schema all at once. One can modify or add to the “current” schema using the appropriate `CREATE`, `DROP`, or `ALTER` statement, e.g., `CREATE TABLE` followed by the declaration of a new table for the schema. We change the “current” schema with a `SET SCHEMA` statement. For example,

```
SET SCHEMA MovieSchema;
```

makes the schema described in Fig. 9.3 the current schema. Then, any declarations of schema elements are added to that schema, and any `DROP` or `ALTER` statements refer to elements already in that schema.

More Schema Elements

Some schema elements that we have not already mentioned, but that occasionally are useful are:

- *Domains*: These are sets of values or simple data types. They are little used today, because object-relational DBMS's provide more powerful type-creation mechanisms; see Section 10.4.
- *Character sets*: These are sets of symbols and methods for encoding them. ASCII and Unicode are common options.
- *Collations*: A collation specifies which characters are "less than" which others. For example, we might use the ordering implied by the ASCII code, or we might treat lower-case and capital letters the same and not compare anything that isn't a letter.
- *Grant statements*: These concern who has access to schema elements. We shall discuss the granting of privileges in Section 10.1.
- *Stored Procedures*: These are executable code; see Section 9.4.

9.2.3 Catalogs

Just as schema elements like tables are created within a schema, schemas are created and modified within a catalog. In principle, we would expect the process of creating and populating catalogs to be analogous to the process of creating and populating schemas. Unfortunately, SQL does not define a standard way to do so, such as a statement

```
CREATE CATALOG <catalog name>
```

followed by a list of schemas belonging to that catalog and the declarations of those schemas.

However, SQL does stipulate a statement

```
SET CATALOG <catalog name>
```

This statement allows us to set the "current" catalog, so new schemas will go into that catalog and schema modifications will refer to schemas in that catalog should there be a name ambiguity.

9.2.4 Clients and Servers in the SQL Environment

A SQL environment is more than a collection of catalogs and schemas. It contains elements whose purpose is to support operations on the database or

Complete Names for Schema Elements

Formally, the name for a schema element such as a table is its catalog name, its schema name, and its own name, connected by dots in that order. Thus, the table *Movies* in the schema *MovieSchema* in the catalog *MovieCatalog* can be referred to as

MovieCatalog.MovieSchema.Movies

If the catalog is the default or current catalog, then we can omit that component of the name. If the schema is also the default or current schema, then that part too can be omitted, and we are left with the element's own name, as is usual. However, we have the option to use the full name if we need to access something outside the current schema or catalog.

databases represented by those catalogs and schemas. According to the SQL standard, within a SQL environment are two special kinds of processes: SQL clients and SQL servers.

In terms of Fig. 9.1, a “SQL server” plays the role what we called a “database server” there. A “SQL client” is like the application servers from that figure. The SQL standard does not define processes analogous to what we called “Web servers” or “clients” in Fig. 9.1.

9.2.5 Connections

If we wish to run some program involving SQL at a host where a SQL client exists, then we may open a connection between the client and server by executing a SQL statement

```
CONNECT TO <server name> AS <connection name>
AUTHORIZATION <name and password>
```

The server name is something that depends on the installation. The word **DEFAULT** can substitute for a name and will connect the user to whatever SQL server the installation treats as the “default server.” We have shown an authorization clause followed by the user's name and password. The latter is the typical method by which a user would be identified to the server, although other strings following **AUTHORIZATION** might be used.

The connection name can be used to refer to the connection later on. The reason we might have to refer to the connection is that SQL allows several connections to be opened by the user, but only one can be active at any time. To switch among connections, we can make **conn1** become the active connection by the statement:

```
SET CONNECTION conn1;
```

Whatever connection was currently active becomes *dormant* until it is reactivated with another `SET CONNECTION` statement that mentions it explicitly.

We also use the name when we drop the connection. We can drop connection `conn1` by

```
DISCONNECT conn1;
```

Now, `conn1` is terminated; it is not dormant and cannot be reactivated.

However, if we shall never need to refer to the connection being created, then `AS` and the connection name may be omitted from the `CONNECT TO` statement. It is also permitted to skip the connection statements altogether. If we simply execute SQL statements at a host with a SQL client, then a default connection will be established on our behalf.

9.2.6 Sessions

The SQL operations that are performed while a connection is active form a *session*. The session lasts as long as the connection that created it. For example, when a connection is made dormant, its session also becomes dormant, and reactivation of the connection by a `SET CONNECTION` statement also makes the session active. Thus, we have shown the session and connection as two aspects of the link between client and server in Fig. 9.4.

Each session has a current catalog and a current schema within that catalog. These may be set with statements `SET SCHEMA` and `SET CATALOG`, as discussed in Sections 9.2.2 and 9.2.3. There is also an authorized user for every session, as we shall discuss in Section 10.1.

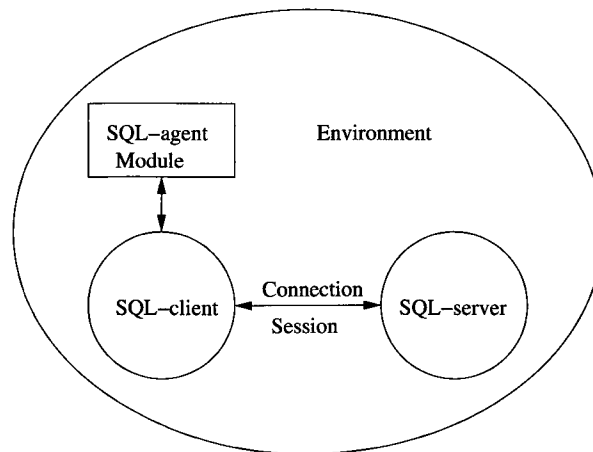


Figure 9.4: The SQL client-server interactions

The Languages of the SQL Standard

Implementations conforming to the SQL standard are required to support at least one of the following seven host languages: ADA, C, Cobol, Fortran, M (formerly called Mumps, and used primarily in the medical community), Pascal, and PL/I. We shall use C in our examples.

9.2.7 Modules

A *module* is the SQL term for an application program. The SQL standard suggests that there are three kinds of modules, but insists only that a SQL implementation offer the user at least one of these types.

1. *Generic SQL Interface.* The user may type SQL statements that are executed by a SQL server. In this mode, each query or other statement is a module by itself. It is this mode that we imagined for most of our examples in this book, although in practice it is rarely used.
2. *Embedded SQL.* This style will be discussed in Section 9.3. Typically, a preprocessor turns the embedded SQL statements into suitable function or procedure calls to the SQL system. The compiled host-language program, including these function calls, is a module.
3. *True Modules.* The most general style of modules envisioned by SQL is a collection of stored functions or procedures, some of which are host-language code and some of which are SQL statements. They communicate among themselves by passing parameters and perhaps via shared variables. PSM modules (Section 9.4) are an example of this type of module.

An execution of a module is called a *SQL agent*. In Fig. 9.4 we have shown both a module and an SQL agent, as one unit, calling upon a SQL client to establish a connection. However, we should remember that the distinction between a module and an SQL agent is analogous to the distinction between a program and a process; the first is code, the second is an execution of that code.

9.3 The SQL/Host-Language Interface

To this point, we have used the *generic SQL interface* in our examples. That is, we have assumed there is a SQL interpreter, which accepts and executes the sorts of SQL queries and commands that we have learned. Although provided as an option by almost all DBMS's, this mode of operation is actually rare. In real systems, such as those described in Section 9.1, there is a program in some

conventional *host* language such as C, but some of the steps in this program are actually SQL statements.

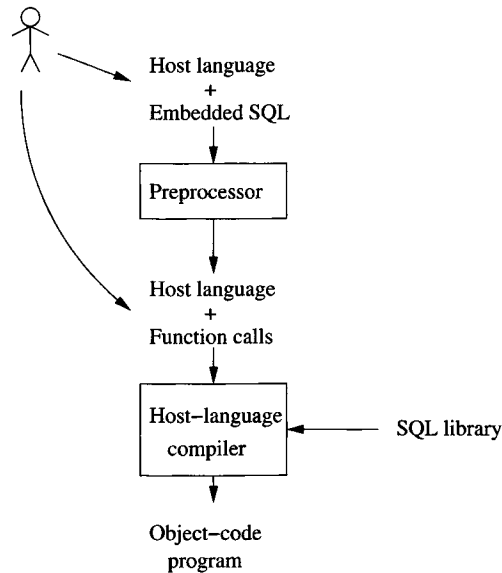


Figure 9.5: Processing programs with SQL statements embedded

A sketch of a typical programming system that involves SQL statements is in Fig. 9.5. There, we see the programmer writing programs in a host language, but with some special “embedded” SQL statements. There are two ways this embedding could take place.

1. *Call-Level Interface.* A library is provided, and the embedding of SQL in the host language is really calls to functions or methods in this library. SQL statements are usually string arguments of these methods. This approach, often referred to as a call-level interface or CLI, is discussed in Section 9.5 and is represented by the curved arrow in Fig. 9.5 from the user directly to the host language.
2. *Directly Embedded SQL.* The entire host-language program, with embedded SQL statements, is sent to a preprocessor, which changes the embedded SQL statements into something that makes sense in the host language. Typically, the SQL statements are replaced by calls to library functions or methods, so the difference between a CLI and direct embedding of SQL is more a matter of “look and feel” than of substance. The preprocessed host-language program is then compiled in the usual manner and operates on the database through execution of the library calls.

In this section, we shall learn the SQL standard for direct embedding in a host language — C in particular. We are also introduced to a number of concepts, such as cursors, that appear in all, or almost all, systems for embedding SQL.

9.3.1 The Impedance Mismatch Problem

The basic problem of connecting SQL statements with those of a conventional programming language is *impedance mismatch*: the fact that the data model of SQL differs so much from the models of other languages. As we know, SQL uses the relational data model at its core. However, C and similar languages use a data model with integers, reals, arithmetic, characters, pointers, record structures, arrays, and so on. Sets are not represented directly in C or these other languages, while SQL does not use pointers, loops and branches, or many other common programming-language constructs. As a result, passing data between SQL and other languages is not straightforward, and a mechanism must be devised to allow the development of programs that use both SQL and another language.

One might first suppose that it is preferable to use a single language. Either do all computation in SQL or forget SQL and do all computation in a conventional language. However, we can dispense with the idea of omitting SQL when there are database operations involved. SQL systems greatly aid the programmer in writing database operations that can be executed efficiently, yet that can be expressed at a very high level. SQL takes from the programmer's shoulders the need to understand how data is organized in storage or how to exploit that storage structure to operate efficiently on the database.

On the other hand, there are many important things that SQL cannot do at all. For example, one cannot write a SQL query to compute n factorial, something that is an easy exercise in C or similar languages.¹ As another example, SQL cannot format its output directly into a convenient form such as a graphic. Thus, real database programming requires both SQL and a host language.

9.3.2 Connecting SQL to the Host Language

When we wish to use a SQL statement within a host-language program, we warn the preprocessor that SQL code is coming with the keywords `EXEC SQL` in front of the statement. We transfer information between the database, which is accessed only by SQL statements, and the host-language program through *shared variables*, which are allowed to appear in both host-language statements

¹We should be careful here. There are extensions to the basic SQL language, such as recursive SQL discussed in Section 10.2 or SQL/PSM discussed in Section 9.4, that do offer “Turing completeness” — the ability to compute anything that can be computed in any other programming language. However, these extensions were never intended for general-purpose calculation, and we do not regard them as general-purpose languages.

and SQL statements. Shared variables are prefixed by a colon within a SQL statement, but they appear without the colon in host-language statements.

A special variable, called `SQLSTATE` in the SQL standard, serves to connect the host-language program with the SQL execution system. The type of `SQLSTATE` is an array of five characters. Each time a function of the SQL library is called, a code is put in the variable `SQLSTATE` that indicates any problems found during that call. The SQL standard also specifies a large number of five-character codes and their meanings.

For example, '00000' (five zeroes) indicates that no error condition occurred, and '02000' indicates that a tuple requested as part of the answer to a SQL query could not be found. The latter code is very important, since it allows us to create a loop in the host-language program that examines tuples from some relation one-at-a-time and to break the loop after the last tuple has been examined.

9.3.3 The DECLARE Section

To declare shared variables, we place their declarations between two embedded SQL statements:

```
EXEC SQL BEGIN DECLARE SECTION;
...
EXEC SQL END DECLARE SECTION;
```

What appears between them is called the *declare section*. The form of variable declarations in the declare section is whatever the host language requires. It only makes sense to declare variables to have types that both the host language and SQL can deal with, such as integers, reals, and character strings or arrays.

Example 9.3: The following statements might appear in a C function that updates the `Studio` relation:

```
EXEC SQL BEGIN DECLARE SECTION;
    char studioName[50], studioAddr[256];
    char SQLSTATE[6];
EXEC SQL END DECLARE SECTION;
```

The first and last statements are the required beginning and end of the declare section. In the middle is a statement declaring two shared variables, `studioName` and `studioAddr`. These are both character arrays and, as we shall see, they can be used to hold a name and address of a studio that are made into a tuple and inserted into the `Studio` relation. The third statement declares `SQLSTATE` to be a six-character array.² □

²We shall use six characters for the five-character value of `SQLSTATE` because in programs to follow we want to use the C function `strcmp` to test whether `SQLSTATE` has a certain value. Since `strcmp` expects strings to be terminated by '\0', we need a sixth character for this endmarker. The sixth character must be set initially to '\0', but we shall not show this assignment in programs to follow.

9.3.4 Using Shared Variables

A shared variable can be used in SQL statements in places where we expect or allow a constant. Recall that shared variables are preceded by a colon when so used. Here is an example in which we use the variables of Example 9.3 as components of a tuple to be inserted into relation *Studio*.

Example 9.4: In Fig. 9.6 is a sketch of a C function `getStudio` that prompts the user for the name and address of a studio, reads the responses, and inserts the appropriate tuple into *Studio*. Lines (1) through (4) are the declarations from Example 9.3. We omit the C code that prints requests and scans entered text to fill the two arrays `studioName` and `studioAddr`.

```
void getStudio() {  
  
    1)      EXEC SQL BEGIN DECLARE SECTION;  
    2)          char studioName[50], studioAddr[256];  
    3)          char SQLSTATE[6];  
    4)      EXEC SQL END DECLARE SECTION;  
  
          /* print request that studio name and address  
            be entered and read response into variables  
            studioName and studioAddr */  
  
    5)      EXEC SQL INSERT INTO Studio(name, address)  
    6)          VALUES (:studioName, :studioAddr);  
}
```

Figure 9.6: Using shared variables to insert a new studio

Then, in lines (5) and (6) is an embedded SQL `INSERT` statement. This statement is preceded by the keywords `EXEC SQL` to indicate that it is indeed an embedded SQL statement rather than ungrammatical C code. The values inserted by lines (5) and (6) are not explicit constants, as they were in all previous examples; rather, the values appearing in line (6) are shared variables whose current values become components of the inserted tuple. $\square$

Any SQL statement that does not return a result (i.e., is not a query) can be embedded in a host-language program by preceding it with `EXEC SQL`. Examples of embeddable SQL statements include insert-, delete-, and update-statements and those statements that create, modify, or drop schema elements such as tables and views.

However, select-from-where queries are not embeddable directly into a host language, because of the “impedance mismatch.” Queries produce bags of tuples as a result, while none of the major host languages support a set or bag

data type directly. Thus, embedded SQL must use one of two mechanisms for connecting the result of queries with a host-language program:

1. *Single-Row SELECT Statements.* A query that produces a single tuple can have that tuple stored in shared variables, one variable for each component of the tuple.
2. *Cursors.* Queries producing more than one tuple can be executed if we declare a *cursor* for the query. The cursor ranges over all tuples in the answer relation, and each tuple in turn can be fetched into shared variables and processed by the host-language program.

We shall consider each of these mechanisms in turn.

9.3.5 Single-Row Select Statements

The form of a single-row select is the same as an ordinary select-from-where statement, except that following the **SELECT** clause is the keyword **INTO** and a list of shared variables. These shared variables each are preceded by a colon, as is the case for all shared variables within a SQL statement. If the result of the query is a single tuple, this tuple's components become the values of these variables. If the result is either no tuple or more than one tuple, then no assignment to the shared variables is made, and an appropriate error code is written in the variable **SQLSTATE**.

Example 9.5: We shall write a C function to read the name of a studio and print the net worth of the studio's president. A sketch of this function is shown in Fig. 9.7. It begins with a declare section, lines (1) through (5), for the variables we shall need. Next, C statements that we do not show explicitly obtain a studio name from the standard input.

Lines (6) through (9) are the single-row select statement. It is quite similar to queries we have already seen. The two differences are that the value of variable **studioName** is used in place of a constant string in the condition of line (9), and there is an **INTO** clause at line (7) that tells us where to put the result of the query. In this case, we expect a single tuple, and tuples have only one component, that for attribute **netWorth**. The value of this one component of one tuple is placed in the shared variable **presNetWorth**. □

9.3.6 Cursors

The most versatile way to connect SQL queries to a host language is with a cursor that runs through the tuples of a relation. This relation can be a stored table, or it can be something that is generated by a query. To create and use a cursor, we need the following statements:

1. A cursor declaration, whose simplest form is:

```

void printNetWorth() {

1)      EXEC SQL BEGIN DECLARE SECTION;
2)      char studioName[50];
3)      int presNetWorth;
4)      char SQLSTATE[6];
5)      EXEC SQL END DECLARE SECTION;

      /* print request that studio name be entered.
         read response into studioName */

6)      EXEC SQL SELECT netWorth
7)          INTO :presNetWorth
8)          FROM Studio, MovieExec
9)          WHERE presC# = cert# AND
              Studio.name = :studioName;

      /* check that SQLSTATE has all 0's and if so, print
         the value of presNetWorth */
}

```

Figure 9.7: A single-row select embedded in a C function

```
EXEC SQL DECLARE <cursor name> CURSOR FOR <query>
```

The query can be either an ordinary select-from-where query or a relation name. The cursor *ranges* over the tuples of the relation produced by the query.

2. A statement `EXEC SQL OPEN`, followed by the cursor name. This statement initializes the cursor to a position where it is ready to retrieve the first tuple of the relation over which the cursor ranges.
3. One or more uses of a *fetch statement*. The purpose of a fetch statement is to get the next tuple of the relation over which the cursor ranges. The fetch statement has the form:

```
EXEC SQL FETCH FROM <cursor name> INTO <list of variables>
```

There is one variable in the list for each attribute of the tuple's relation. If there is a tuple available to be fetched, these variables are assigned the values of the corresponding components from that tuple. If the tuples have been exhausted, then no tuple is returned, and the value of `SQLSTATE` is set to '02000', a code that means "no tuple found."

4. The statement `EXEC SQL CLOSE` followed by the name of the cursor. This statement closes the cursor, which now no longer ranges over tuples of the relation. It can, however, be reinitialized by another `OPEN` statement, in which case it ranges anew over the tuples of this relation.

Example 9.6: Suppose we wish to determine the number of movie executives whose net worths fall into a sequence of bands of exponentially growing size, each band corresponding to a number of digits in the net worth. We shall design a query that retrieves the `netWorth` field of all the `MovieExec` tuples into a shared variable called `worth`. A cursor called `execCursor` will range over all these one-component tuples. Each time a tuple is fetched, we compute the number of digits in the integer `worth` and increment the appropriate element of an array `counts`.

The C function `worthRanges` begins in line (1) of Fig. 9.8. Line (2) declares some variables used only by the C function, not by the embedded SQL. The array `counts` holds the counts of executives in the various bands, `digits` counts the number of digits in a net worth, and `i` is an index ranging over the elements of array `counts`.

Lines (3) through (6) are a SQL declare section in which shared variable `worth` and the usual `SQLSTATE` are declared. Lines (7) and (8) declare `execCursor` to be a cursor that ranges over the values produced by the query on line (8). This query simply asks for the `netWorth` components of all the tuples in `MovieExec`. This cursor is then opened at line (9). Line (10) completes the initialization by zeroing the elements of array `counts`.

The main work is done by the loop of lines (11) through (16). At line (12) a tuple is fetched into shared variable `worth`. Since tuples produced by the query of line (8) have only one component, we need only one shared variable, although in general there would be as many variables as there are components of the retrieved tuples. Line (13) tests whether the fetch has been successful. Here, we use a macro `NO_MORE_TUPLES`, defined by

```
#define NO_MORE_TUPLES !(strcmp(SQLSTATE,"02000"))
```

Recall that "02000" is the `SQLSTATE` code that means no tuple was found. If there are no more tuples, we break out of the loop and go to line (17).

If a tuple has been fetched, then at line (14) we initialize the number of digits in the net worth to 1. Line (15) is a loop that repeatedly divides the net worth by 10 and increments `digits` by 1. When the net worth reaches 0 after division by 10, `digits` holds the correct number of digits in the value of `worth` that was originally retrieved. Finally, line (16) increments the appropriate element of the array `counts` by 1. We assume that the number of digits is no more than 14. However, should there be a net worth with 15 or more digits, line (16) will not increment any element of the `counts` array, since there is no appropriate range; i.e., enormous net worths are thrown away and do not affect the statistics.

Line (17) begins the wrap-up of the function. The cursor is closed, and lines (18) and (19) print the values in the `counts` array. □

```

1) void worthRanges() {

2)     int i, digits, counts[15];
3)     EXEC SQL BEGIN DECLARE SECTION;
4)         int worth;
5)         char SQLSTATE[6];
6)     EXEC SQL END DECLARE SECTION;
7)     EXEC SQL DECLARE execCursor CURSOR FOR
8)         SELECT netWorth FROM MovieExec;

9)     EXEC SQL OPEN execCursor;
10)    for(i=1; i<15; i++) counts[i] = 0;
11)    while(1) {
12)        EXEC SQL FETCH FROM execCursor INTO :worth;
13)        if(NO_MORE_TUPLES) break;
14)        digits = 1;
15)        while((worth /= 10) > 0) digits++;
16)        if(digits <= 14) counts[digits]++;
17)    }
18)    EXEC SQL CLOSE execCursor;
19)    for(i=0; i<15; i++)
20)        printf("digits = %d: number of execs = %d\n",
21)            i, counts[i]);
22}

```

Figure 9.8: Grouping executive net worths into exponential bands

9.3.7 Modifications by Cursor

When a cursor ranges over the tuples of a base table (i.e., a relation that is stored in the database), then one can not only read the current tuple, but one can update or delete the current tuple. The syntax of these **UPDATE** and **DELETE** statements are the same as we encountered in Section 6.5, with the exception of the **WHERE** clause. That clause may only be **WHERE CURRENT OF** followed by the name of the cursor. Of course it is possible for the host-language program reading the tuple to apply whatever condition it likes to the tuple before deciding whether or not to delete or update it.

Example 9.7: In Fig. 9.9 we see a C function that looks at each tuple of **MovieExec** and decides either to delete the tuple or to double the net worth. In lines (3) and (4) we declare variables that correspond to the four attributes of **MovieExec**, as well as the necessary **SQLSTATE**. Then, at line (6), **execCursor** is declared to range over the stored relation **MovieExec** itself.

Lines (8) through (14) are the loop, in which the cursor **execCursor** refers to each tuple of **MovieExec**, in turn. Line (9) fetches the current tuple into

```

1) void changeWorth() {

2)     EXEC SQL BEGIN DECLARE SECTION;
3)         int certNo, worth;
4)         char execName[31], execAddr[256], SQLSTATE[6];
5)     EXEC SQL END DECLARE SECTION;
6)     EXEC SQL DECLARE execCursor CURSOR FOR MovieExec;

7)     EXEC SQL OPEN execCursor;
8)     while(1) {
9)         EXEC SQL FETCH FROM execCursor INTO :execName,
           :execAddr, :certNo, :worth;
10)        if(NO_MORE_TUPLES) break;
11)        if (worth < 1000)
12)            EXEC SQL DELETE FROM MovieExec
                WHERE CURRENT OF execCursor;
13)        else
14)            EXEC SQL UPDATE MovieExec
                SET netWorth = 2 * netWorth
                WHERE CURRENT OF execCursor;
15)    }
    EXEC SQL CLOSE execCursor;
}

```

Figure 9.9: Modifying executive net worths

the four variables used for this purpose; note that only `worth` is actually used. Line (10) tests whether we have exhausted the tuples of `MovieExec`. We have again used the macro `NO_MORE_TUPLES` for the condition that variable `SQLSTATE` has the “no more tuples” code “02000”.

In the test of line (11) we ask if the net worth is under \$1000. If so, the tuple is deleted by the `DELETE` statement of line (12). Note that the `WHERE` clause refers to the cursor, so the current tuple of `MovieExec`, the one we just fetched, is deleted from `MovieExec`. If the net worth is at least \$1000, then at line (14), the net worth in the same tuple is doubled, instead. □

9.3.8 Protecting Against Concurrent Updates

Suppose that as we examine the net worths of movie executives using the function `worthRanges` of Fig. 9.8, some other process is modifying the underlying `MovieExec` relation. What should we do about this possibility? Perhaps nothing. We might be happy with approximate statistics, and we don’t care whether or not we count an executive who was in the process of being deleted, for example. Then, we simply accept what tuples we get through the cursor.

However, we may not wish to allow concurrent changes to affect the tuples we see through this cursor. Rather, we may insist on the statistics being taken on the relation as it exists at some point in time. In terms of the transactions of Section 6.6, we want the code that runs the cursor through the relation to be serializable with any other operations on the relation. To obtain this guarantee, we may declare the cursor *insensitive* to concurrent changes.

Example 9.8: We could modify lines (7) and (8) of Fig. 9.8 to be:

```

7)      EXEC SQL DECLARE execCursor INSENSITIVE CURSOR FOR
8)      SELECT netWorth FROM MovieExec;
```

If `execCursor` is so declared, then the SQL system will guarantee that changes to relation `MovieExec` made between one opening and closing of `execCursor` will not affect the set of tuples fetched. $\square$

There are certain cursors ranging over a relation R about which we may say with certainty that they will not change R . Such a cursor can run simultaneously with an insensitive cursor for R , without risk of changing the relation R that the insensitive cursor sees. If we declare a cursor `FOR READ ONLY`, then the database system can be sure that the underlying relation will not be modified because of access to the relation through this cursor.

Example 9.9: We could append after line (8) of `worthRanges` in Fig. 9.8 a line

```
FOR READ ONLY;
```

If so, then any attempt to execute a modification through cursor `execCursor` would cause an error. $\square$

9.3.9 Dynamic SQL

Our model of SQL embedded in a host language has been that of specific SQL queries and commands within a host-language program. An alternative style of embedded SQL has the statements themselves be computed by the host language. Such statements are not known at compile time, and thus cannot be handled by a SQL preprocessor or a host-language compiler.

An example of such a situation is a program that prompts the user for an SQL query, reads the query, and then executes that query. The generic interface for ad-hoc SQL queries that we assumed in Chapter 6 is an example of just such a program. If queries are read and executed at run-time, there is nothing that can be done at compile-time. The query has to be parsed and a suitable way to execute the query found by the SQL system, immediately after the query is read.

The host-language program must instruct the SQL system to take the character string just read, to turn it into an executable SQL statement, and finally to execute that statement. There are two *dynamic SQL* statements that perform these two steps.

1. **EXEC SQL PREPARE *V* FROM <expression>**, where *V* is a SQL variable. The expression can be any host-language expression whose value is a string; this string is treated as a SQL statement. Presumably, the SQL statement is parsed and a good way to execute it is found by the SQL system, but the statement is not executed. Rather, the plan for executing the SQL statement becomes the value of *V*.
2. **EXEC SQL EXECUTE *V***. This statement causes the SQL statement denoted by variable *V* to be executed.

Both steps can be combined into one, with the statement:

EXEC SQL EXECUTE IMMEDIATE <expression>

The disadvantage of combining these two parts is seen if we prepare a statement once and then execute it many times. With **EXECUTE IMMEDIATE** the cost of preparing the statement is paid each time the statement is executed, rather than paid only once, when we prepare it.

Example 9.10: In Fig. 9.10 is a sketch of a C program that reads text from standard input into a variable `query`, prepares it, and executes it. The SQL variable `SQLquery` holds the prepared query. Since the query is only executed once, the line:

EXEC SQL EXECUTE IMMEDIATE :query;

could replace lines (6) and (7) of Fig. 9.10. □

```

1) void readQuery() {
2)     EXEC SQL BEGIN DECLARE SECTION;
3)     char *query;
4)     EXEC SQL END DECLARE SECTION;
5)     /* prompt user for a query, allocate space (e.g.,
        use malloc) and make shared variable :query point
        to the first character of the query */
6)     EXEC SQL PREPARE SQLquery FROM :query;
7)     EXEC SQL EXECUTE SQLquery;
    }

```

Figure 9.10: Preparing and executing a dynamic SQL query

9.3.10 Exercises for Section 9.3

Exercise 9.3.1: Write the following embedded SQL queries, based on the database schema

```
Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)
```

of Exercise 2.4.1. You may use any host language with which you are familiar, and details of host-language programming may be replaced by clear comments if you wish.

- a) Ask the user for a price and find the PC whose price is closest to the desired price. Print the maker, model number, and speed of the PC.
- b) Ask the user for minimum values of the speed, RAM, hard-disk size, and screen size that they will accept. Find all the laptops that satisfy these requirements. Print their specifications (all attributes of **Laptop**) and their manufacturer.
- ! c) Ask the user for a manufacturer. Print the specifications of all products by that manufacturer. That is, print the model number, product-type, and all the attributes of whichever relation is appropriate for that type.
- !! d) Ask the user for a “budget” (total price of a PC and printer), and a minimum speed of the PC. Find the cheapest “system” (PC plus printer) that is within the budget and minimum speed, but make the printer a color printer if possible. Print the model numbers for the chosen system.
- e) Ask the user for a manufacturer, model number, speed, RAM, hard-disk size, and price of a new PC. Check that there is no PC with that model number. Print a warning if so, and otherwise insert the information into tables **Product** and **PC**.

Exercise 9.3.2: Write the following embedded SQL queries, based on the database schema

```
Classes(class, type, country, numGuns, bore, displacement)
Ships(name, class, launched)
Battles(name, date)
Outcomes(ship, battle, result)
```

of Exercise 2.4.3.

- a) The firepower of a ship is roughly proportional to the number of guns times the cube of the bore of the guns. Find the class with the largest firepower.

- ! b) Ask the user for the name of a battle. Find the countries of the ships involved in the battle. Print the country with the most ships sunk and the country with the most ships damaged.
- c) Ask the user for the name of a class and the other information required for a tuple of table `Classes`. Then ask for a list of the names of the ships of that class and their dates launched. However, the user need not give the first name, which will be the name of the class. Insert the information gathered into `Classes` and `Ships`.
- ! d) Examine the `Battles`, `Outcomes`, and `Ships` relations for ships that were in battle before they were launched. Prompt the user when there is an error found, offering the option to change the date of launch or the date of the battle. Make whichever change is requested.

9.4 Stored Procedures

In this section, we introduce you to *Persistent, Stored Modules* (SQL/PSM, or just PSM). PSM is part of the latest revision to the SQL standard, called SQL:2003. It allows us to write procedures in a simple, general-purpose language and to store them in the database, as part of the schema. We can then use these procedures in SQL queries and other statements to perform computations that cannot be done with SQL alone. Each commercial DBMS offers its own extension of PSM. In this book, we shall describe the SQL/PSM standard, which captures the major ideas of these facilities, and which should help you understand the language associated with any particular system. References to PSM extensions provided with several major commercial systems are in the bibliographic notes.

9.4.1 Creating PSM Functions and Procedures

In PSM, you define *modules*, which are collections of function and procedure definitions, temporary relation declarations, and several other optional declarations. The major elements of a procedure declaration are:

```
CREATE PROCEDURE <name> (<parameters>)
    <local declarations>
    <procedure body>;
```

This form should be familiar from a number of programming languages; it consists of a procedure name, a parenthesized list of parameters, some optional local-variable declarations, and the executable body of code that defines the procedure. A function is defined in almost the same way, except that the keyword `FUNCTION` is used, and there is a return-value type that must be specified. That is, the elements of a function definition are:

```
CREATE FUNCTION <name> (<parameters>) RETURNS <type>
    <local declarations>
    <function body>;
```

The parameters of a PSM procedure are mode-name-type triples. That is, the parameter name is not only followed by its declared type, as usual in programming languages, but it is preceded by a “mode,” which is either IN, OUT, or INOUT. These three keywords indicate that the parameter is input-only, output-only, or both input and output, respectively. IN is the default, and can be omitted.

Function parameters, on the other hand, may only be of mode IN. That is, PSM forbids side-effects in functions, so the only way to obtain information from a function is through its return-value. We shall not specify the IN mode for function parameters, although we do so in procedure definitions.

Example 9.11: While we have not yet learned the variety of statements that can appear in procedure and function bodies, one kind should not surprise us: an SQL statement. The limitation on these statements is the same as for embedded SQL, as we introduced in Section 9.3.4: only single-row-select statements and cursor-based accesses are permitted as queries. In Fig. 9.11 is a PSM procedure that takes two addresses — an old address and a new address — as parameters and replaces the old address by the new everywhere it appears in *MovieStar*.

```
1) CREATE PROCEDURE Move(
2)     IN oldAddr VARCHAR(255),
3)     IN newAddr VARCHAR(255)
4) )
5) UPDATE MovieStar
6) SET address = newAddr
   WHERE address = oldAddr;
```

Figure 9.11: A procedure to change addresses

Line (1) introduces the procedure and its name, *Move*. Lines (2) and (3) declare two input parameters, both of whose types are *VARCHAR(255)*. This type is consistent with the type we declared for the attribute *address* of *MovieStar* in Fig. 2.8. Lines (4) through (6) are a conventional *UPDATE* statement. However, notice that the parameter names can be used as if they were constants. Unlike host-language variables, which require a colon prefix when used in SQL (see Section 9.3.2), parameters and other local variables of PSM procedures and functions require no colon. □

9.4.2 Some Simple Statement Forms in PSM

Let us begin with a potpourri of statement forms that are easy to master.

1. *The call-statement:* The form of a procedure call is:

CALL <procedure name> (<argument list>);

That is, the keyword **CALL** is followed by the name of the procedure and a parenthesized list of arguments, as in most any language. This call can, however, be made from a variety of places:

- i.* From a host-language program, in which it might appear as

EXEC SQL CALL Foo(:x, 3);

for instance.

- ii.* As a statement of another PSM function or procedure.
- iii.* As a SQL command issued to the generic SQL interface. For example, we can issue a statement such as

CALL Foo(1, 3);

to such an interface, and have stored procedure **Foo** executed with its two parameters set equal to 1 and 3, respectively.

Note that it is not permitted to call a function. You invoke functions in PSM as you do in C: use the function name and suitable arguments as part of an expression.

2. *The return-statement:* Its form is

RETURN <expression>;

This statement can only appear in a function. It evaluates the expression and sets the return-value of the function equal to that result. However, at variance with common programming languages, the return-statement of PSM does *not* terminate the function. Rather, control continues with the following statement, and it is possible that the return-value will be changed before the function completes.

3. *Declarations of local variables:* The statement form

DECLARE <name> <type>;

declares a variable with the given name to have the given type. This variable is local, and its value is not preserved by the DBMS after a running of the function or procedure. Declarations must precede executable statements in the function or procedure body.

4. *Assignment Statements:* The form of an assignment is:

SET <variable> = <expression>;

Except for the introductory keyword SET, assignment in PSM is quite like assignment in other languages. The expression on the right of the equal-sign is evaluated, and its value becomes the value of the variable on the left. NULL is a permissible expression. The expression may even be a query, as long as it returns a single value.

5. *Statement groups*: We can form a list of statements ended by semicolons and surrounded by keywords BEGIN and END. This construct is treated as a single statement and can appear anywhere a single statement can. In particular, since a procedure or function body is expected to be a single statement, we can put any sequence of statements in the body by surrounding them by BEGIN...END.
6. *Statement labels*: We label a statement by prefixing it with a name (the label) and a colon.

9.4.3 Branching Statements

For our first complex PSM statement type, let us consider the if-statement. The form is only a little strange; it differs from C or similar languages in that:

1. The statement ends with keywords END IF.
2. If-statements nested within the else-clause are introduced with the single word ELSEIF.

Thus, the general form of an if-statement is as suggested by Fig. 9.12. The condition is any boolean-valued expression, as can appear in the WHERE clause of SQL statements. Each statement list consists of statements ended by semicolons, but does not need a surrounding BEGIN...END. The final ELSE and its statement(s) are optional; i.e., IF...THEN...END IF alone or with ELSEIF's is acceptable.

Example 9.12: Let us write a function to take a year y and a studio s , and return a boolean that is TRUE if and only if studio s produced at least one comedy in year y or did not produce any movies at all in that year. The code appears in Fig. 9.13.

Line (1) introduces the function and includes its arguments. We do not need to specify a mode for the arguments, since that can only be IN for a function. Lines (2) and (3) test for the case where there are no movies at all by studio s in year y , in which case we set the return-value to TRUE at line (4). Note that line (4) does not cause the function to return. Technically, it is the flow of control dictated by the if-statements that causes control to jump from line (4) to line (9), where the function completes and returns.

```

IF <condition> /tt THEN
    <statement list>
ELSEIF <condition> /tt THEN
    <statement list>
ELSEIF
    ...
ELSE
    <statement list>
END IF;

```

Figure 9.12: The form of an if-statement

```

1) CREATE FUNCTION BandW(y INT, s CHAR(15)) RETURNS BOOLEAN
2) IF NOT EXISTS(
3)     SELECT * FROM Movies WHERE year = y AND
        studioName = s)
4) THEN RETURN TRUE;
5) ELSEIF 1 <=
6)     (SELECT COUNT(*) FROM Movies WHERE year = y AND
        studioName = s AND genre = 'comedy')
7) THEN RETURN TRUE;
8) ELSE RETURN FALSE;
9) END IF;

```

Figure 9.13: If there are any movies at all, then at least one has to be a comedy

If studio s made movies in year y , then lines (5) and (6) test if at least one of them was a comedy. If so, the return-value is again set to true, this time at line (7). In the remaining case, studio s made movies but only in color, so we set the return-value to FALSE at line (8). $\square$

9.4.4 Queries in PSM

There are several ways that select-from-where queries are used in PSM.

1. Subqueries can be used in conditions, or in general, any place a subquery is legal in SQL. We saw two examples of subqueries in lines (3) and (6) of Fig. 9.13, for instance.
2. Queries that return a single value can be used as the right sides of assignment statements.
3. A single-row select statement is a legal statement in PSM. Recall this statement has an INTO clause that specifies variables into which the com-

ponents of the single returned tuple are placed. These variables could be local variables or parameters of a PSM procedure. The general form was discussed in the context of embedded SQL in Section 9.3.5.

4. We can declare and use a cursor, essentially as it was described in Section 9.3.6 for embedded SQL. The declaration of the cursor, `OPEN`, `FETCH`, and `CLOSE` statements are all as described there, with the exceptions that:
 - (a) No `EXEC SQL` appears in the statements, and
 - (b) The variables do not use a colon prefix.

```
CREATE PROCEDURE SomeProc(IN studioName CHAR(15))

DECLARE presNetWorth INTEGER;

SELECT netWorth
  INTO presNetWorth
  FROM Studio, MovieExec
 WHERE presC# = cert# AND Studio.name = studioName;
  ...
```

Figure 9.14: A single-row select in PSM

Example 9.13: In Fig. 9.14 is the single-row select of Fig. 9.7, redone for PSM and placed in the context of a hypothetical procedure definition. Note that, because the single-row select returns a one-component tuple, we could also get the same effect from an assignment statement, as:

```
SET presNetWorth = (SELECT netWorth
  FROM Studio, MovieExec
 WHERE presC# = cert# AND Studio.name = studioName);
```

We shall defer examples of cursor use until we learn the PSM loop statements in the next section. □

9.4.5 Loops in PSM

The basic loop construct in PSM is:

```
LOOP
  <statement list>
END LOOP;
```

One often labels the `LOOP` statement, so it is possible to break out of the loop, using a statement:

```
LEAVE <loop label>;
```

In the common case that the loop involves the fetching of tuples via a cursor, we often wish to leave the loop when there are no more tuples. It is useful to declare a *condition* name for the SQLSTATE value that indicates no tuple found ('02000', recall); we do so with:

```
DECLARE Not_Found CONDITION FOR SQLSTATE '02000';
```

More generally, we can declare a condition with any desired name corresponding to any SQLSTATE value by

```
DECLARE <name> CONDITION FOR SQLSTATE <value>;
```

We are now ready to take up an example that ties together cursor operations and loops in PSM.

Example 9.14: Figure 9.15 shows a PSM procedure that takes a studio name *s* as an input argument and produces in output arguments **mean** and **variance** the mean and variance of the lengths of all the movies owned by studio *s*. Lines (1) through (4) declare the procedure and its parameters.

Lines (5) through (8) are local declarations. We define **Not_Found** to be the name of the condition that means a **FETCH** failed to return a tuple at line (5). Then, at line (6), the cursor **MovieCursor** is defined to return the set of the lengths of the movies by studio *s*. Lines (7) and (8) declare two local variables that we'll need. Integer **newLength** holds the result of a **FETCH**, while **movieCount** counts the number of movies by studio *s*. We need **movieCount** so that, at the end, we can convert a sum of lengths into an average (mean) of lengths and a sum of squares of the lengths into a variance.

The rest of the lines are the body of the procedure. We shall use **mean** and **variance** as temporary variables, as well as for "returning" the results at the end. In the major loop, **mean** actually holds the sum of the lengths, and **variance** actually holds the sum of the squares of the lengths. Thus, lines (9) through (11) initialize these variables and the count of the movies to 0. Line (12) opens the cursor, and lines (13) through (19) form the loop labeled **movieLoop**.

Line (14) performs a fetch, and at line (15) we check that another tuple was found. If not, we leave the loop. Lines (16) through (18) accumulate values; we add 1 to **movieCount**, add the length to **mean** (which, recall, is really computing the sum of lengths), and we add the square of the length to **variance**.

When all movies by studio *s* have been seen, we leave the loop, and control passes to line (20). At that line, we turn **mean** into its correct value by dividing the sum of lengths by the count of movies. At line (21), we make **variance** truly hold the variance by dividing the sum of squares of the lengths by the number of movies and subtracting the square of the mean. See Exercise 9.4.4 for a discussion of why this calculation is correct. Line (22) closes the cursor, and we are done. $\square$

```

1) CREATE PROCEDURE MeanVar(
2)     IN s CHAR(15),
3)     OUT mean REAL,
4)     OUT variance REAL
5) )
6) DECLARE Not_Found CONDITION FOR SQLSTATE '02000';
7) DECLARE MovieCursor CURSOR FOR
8)     SELECT length FROM Movies WHERE studioName = s;
9) DECLARE newLength INTEGER;
10) DECLARE movieCount INTEGER;
11)
12) BEGIN
13)     SET mean = 0.0;
14)     SET variance = 0.0;
15)     SET movieCount = 0;
16)     OPEN MovieCursor;
17)     movieLoop: LOOP
18)         FETCH FROM MovieCursor INTO newLength;
19)         IF Not_Found THEN LEAVE movieLoop END IF;
20)         SET movieCount = movieCount + 1;
21)         SET mean = mean + newLength;
22)         SET variance = variance + newLength * newLength;
23)     END LOOP;
24)     SET mean = mean/movieCount;
25)     SET variance = variance/movieCount - mean * mean;
26)     CLOSE MovieCursor;
27) END;

```

Figure 9.15: Computing the mean and variance of lengths of movies by one studio

9.4.6 For-Loops

There is also in PSM a for-loop construct, but it is used only to iterate over a cursor. The form of the statement is shown in Fig. 9.16. This statement not only declares a cursor, but it handles for us a number of “grubby details”: the opening and closing of the cursor, the fetching, and the checking whether there are no more tuples to be fetched. However, since we are not fetching tuples for ourselves, we can not specify the variable(s) into which component(s) of a tuple are placed. Thus, the names used for the attributes in the result of the query are also treated by PSM as local variables of the same type.

Example 9.15: Let us redo the procedure of Fig. 9.15 using a for-loop. The code is shown in Fig. 9.17. Many things have not changed. The declaration of the procedure in lines (1) through (4) of Fig. 9.17 are the same, as is the

Other Loop Constructs

PSM also allows while- and repeat-loops, which have the expected meaning, as in C. That is, we can create a loop of the form

```
WHILE <condition> DO
  <statement list>
END WHILE;
```

or a loop of the form

```
REPEAT
  <statement list>
UNTIL <condition>
END REPEAT;
```

Incidentally, if we label these loops, or the loop formed by a loop-statement or for-statement, then we can place the label as well after the `END LOOP` or other ender. The advantage of doing so is that it makes clearer where each loop ends, and it allows the PSM compiler to catch some syntactic errors involving the omission of an `END`.

```
FOR <loop name> AS <cursor name> CURSOR FOR
  <query>
DO
  <statement list>
END FOR;
```

Figure 9.16: The PSM for-statement

declaration of local variable `movieCount` at line (5).

However, we no longer need to declare a cursor in the declaration portion of the procedure, and we do not need to define the condition `Not_Found`. Lines (6) through (8) initialize the variables, as before. Then, in line (9) we see the for-loop, which also defines the cursor `MovieCursor`. Lines (11) through (13) are the body of the loop. Notice that in lines (12) and (13), we refer to the length retrieved via the cursor by the attribute name `length`, rather than by the local variable name `newLength`, which does not exist in this version of the procedure. Lines (15) and (16) compute the correct values for the output variables, exactly as in the earlier version of this procedure. □

```

1) CREATE PROCEDURE MeanVar(
2)     IN s CHAR(15),
3)     OUT mean REAL,
4)     OUT variance REAL
5) )
6) DECLARE movieCount INTEGER;
7)
8) BEGIN
9)     SET mean = 0.0;
10)    SET variance = 0.0;
11)    SET movieCount = 0;
12)    FOR movieLoop AS MovieCursor CURSOR FOR
13)        SELECT length FROM Movies WHERE studioName = s;
14)    DO
15)        SET movieCount = movieCount + 1;
16)        SET mean = mean + length;
17)        SET variance = variance + length * length;
18)    END FOR;
19)    SET mean = mean/movieCount;
20)    SET variance = variance/movieCount - mean * mean;
21) END;

```

Figure 9.17: Computing the mean and variance of lengths using a for-loop

9.4.7 Exceptions in PSM

A SQL system indicates error conditions by setting a nonzero sequence of digits in the five-character string `SQLSTATE`. We have seen one example of these codes: '02000' for "no tuple found." For another example, '21000' indicates that a single-row select has returned more than one row.

PSM allows us to declare a piece of code, called an *exception handler*, that is invoked whenever one of a list of these error codes appears in `SQLSTATE` during the execution of a statement or list of statements. Each exception handler is associated with a block of code, delineated by `BEGIN...END`. The handler appears within this block, and it applies only to statements within the block.

The components of the handler are:

1. A list of exception conditions that invoke the handler when raised.
2. Code to be executed when one of the associated exceptions is raised.
3. An indication of where to go after the handler has finished its work.

The form of a handler declaration is:

```

DECLARE <where to go next> HANDLER FOR <condition list>
    <statement>

```

Why Do We Need Names in For-Loops?

Notice that `movieLoop` and `MovieCursor`, although declared at line (9) of Fig. 9.17, are never used in that procedure. Nonetheless, we have to invent names, both for the for-loop itself and for the cursor over which it iterates. The reason is that the PSM interpreter will translate the for-loop into a conventional loop, much like the code of Fig. 9.15, and in this code, there is a need for both names.

The choices for “where to go” are:

- a) **CONTINUE**, which means that after executing the statement in the handler declaration, we execute the statement after the one that raised the exception.
- b) **EXIT**, which means that after executing the handler’s statement, control leaves the **BEGIN . . . END** block in which the handler is declared. The statement after this block is executed next.
- c) **UNDO**, which is the same as **EXIT**, except that any changes to the database or local variables that were made by the statements of the block executed so far are undone. That is, the block is a transaction, which is aborted by the exception.

The “condition list” is a comma-separated list of conditions, which are either declared conditions, like `Not_Found` in line (5) of Fig. 9.15, or expressions of the form `SQLSTATE` and a five-character string.

Example 9.16: Let us write a PSM function that takes a movie title as argument and returns the year of the movie. If there is no movie of that title or more than one movie of that title, then `NULL` must be returned. The code is shown in Fig. 9.18.

Lines (2) and (3) declare symbolic conditions; we do not have to make these definitions, and could as well have used the SQL states for which they stand in line (4). Lines (4), (5), and (6) are a block, in which we first declare a handler for the two conditions in which either zero tuples are returned, or more than one tuple is returned. The action of the handler, on line (5), is simply to set the return-value to `NULL`.

Line (6) is the statement that does the work of the function `GetYear`. It is a **SELECT** statement that is expected to return exactly one integer, since that is what the function `GetYear` returns. If there is exactly one movie with title *t* (the input parameter of the function), then this value will be returned. However, if an exception is raised at line (6), either because there is no movie with title *t* or several movies with that title, then the handler is invoked, and `NULL` instead

```

1) CREATE FUNCTION GetYear(t VARCHAR(255)) RETURNS INTEGER

2) DECLARE Not_Found CONDITION FOR SQLSTATE '02000';
3) DECLARE Too_Many CONDITION FOR SQLSTATE '21000';

BEGIN

4)     DECLARE EXIT HANDLER FOR Not_Found, Too_Many
5)         RETURN NULL;
6)     RETURN (SELECT year FROM Movies WHERE title = t);

END;
```

Figure 9.18: Handling exceptions in which a single-row select returns other than one tuple

becomes the return-value. Also, since the handler is an EXIT handler, control next passes to the point after the END. Since that point is the end of the function, *GetYear* returns at that time, with the return-value NULL. □

9.4.8 Using PSM Functions and Procedures

As we mentioned in Section 9.4.2, we can call a PSM procedure anywhere SQL statements can appear, e.g., as embedded SQL, from PSM code itself, or from SQL issued to the generic interface. We invoke a procedure by preceding it by the keyword CALL. In addition, a PSM function can be used as part of an expression, e.g., in a WHERE clause. Here is an example of how a function can be used within an expression.

Example 9.17: Suppose that our schema includes a module with the function *GetYear* of Fig. 9.18. Imagine that we are sitting at the generic interface, and we want to enter the fact that Denzel Washington was a star of *Remember the Titans*. However, we forget the year in which that movie was made. As long as there was only one movie of that name, and it is in the *Movies* relation, we don't have to look it up in a preliminary query. Rather, we can issue to the generic SQL interface the following insertion:

```

INSERT INTO StarsIn(movieTitle, movieYear, starName)
VALUES('Remember the Titans', GetYear('Remember the Titans'),
      'Denzel Washington');
```

Since *GetYear* returns NULL if there is not a unique movie by the name of *Remember the Titans*, it is possible that this insertion will have NULL in the middle component. □

9.4.9 Exercises for Section 9.4

Exercise 9.4.1: Using our running movie database:

```

Movies(title, year, length, genre, studioName, producerC#)
StarsIn(movieTitle, movieYear, starName)
MovieStar(name, address, gender, birthdate)
MovieExec(name, address, cert#, netWorth)
Studio(name, address, presC#)

```

write PSM procedures or functions to perform the following tasks:

- a) Given the name of a movie studio, produce the net worth of its president.
- b) Given a name and address, return 1 if the person is a movie star but not an executive, 2 if the person is an executive but not a star, 3 if both, and 4 if neither.
- ! c) Given a studio name, assign to output parameters the titles of the two longest movies by that studio. Assign NULL to one or both parameters if there is no such movie (e.g., if there is only one movie by a studio, there is no “second-longest”).
- ! d) Given a star name, find the earliest (lowest year) movie of more than 120 minutes length in which they appeared. If there is no such movie, return the year 0.
- e) Given an address, find the name of the unique star with that address if there is exactly one, and return NULL if there is none or more than one.
- f) Given the name of a star, delete them from `MovieStar` and delete all their movies from `StarsIn` and `Movies`.

Exercise 9.4.2: Write the following PSM functions or procedures, based on the database schema

```

Product(maker, model, type)
PC(model, speed, ram, hd, price)
Laptop(model, speed, ram, hd, screen, price)
Printer(model, color, type, price)

```

of Exercise 2.4.1.

- a) Take a price as argument and return the model number of the PC whose price is closest.
- b) Take a maker and model as arguments, and return the price of whatever type of product that model is.
- ! c) Take model, speed, ram, hard-disk, and price information as arguments, and insert this information into the relation `PC`. However, if there is already a PC with that model number (tell by assuming that violation of a key constraint on insertion will raise an exception with `SQLSTATE` equal to `'23000'`), then keep adding 1 to the model number until you find a model number that is not already a PC model number.

- ! d) Given a price, produce the number of PC's, the number of laptops, and the number of printers selling for more than that price.

Exercise 9.4.3: Write the following PSM functions or procedures, based on the database schema

```
Classes(class, type, country, numGuns, bore, displacement)
Ships(name, class, launched)
Battles(name, date)
Outcomes(ship, battle, result)
```

of Exercise 2.4.3.

- a) The firepower of a ship is roughly proportional to the number of guns times the cube of the bore. Given a class, find its firepower.
 - ! b) Given the name of a battle, produce the two countries whose ships were involved in the battle. If there are more or fewer than two countries involved, produce NULL for both countries.
 - c) Take as arguments a new class name, type, country, number of guns, bore, and displacement. Add this information to **Classes** and also add the ship with the class name to **Ships**.
 - ! d) Given a ship name, determine if the ship was in a battle with a date before the ship was launched. If so, set the date of the battle and the date the ship was launched to 0.
- ! **Exercise 9.4.4:** In Fig. 9.15, we used a tricky formula for computing the variance of a sequence of numbers $x_1, x_2, \dots, x_n$. Recall that the variance is the average square of the deviation of these numbers from their mean. That is, the variance is $(\sum_{i=1}^n (x_i - \bar{x})^2)/n$, where the mean $\bar{x}$ is $(\sum_{i=1}^n x_i)/n$. Prove that the formula for the variance used in Fig. 9.15, which is

$$\left(\sum_{i=1}^n (x_i)^2\right)/n - \left((\sum_{i=1}^n x_i)/n\right)^2$$

yields the same value.

9.5 Using a Call-Level Interface

When using a *call-level interface* (CLI), we write ordinary host-language code, and we use a library of functions that allow us to connect to and access a database, passing SQL statements to that database. The differences between this approach and embedded SQL programming are, in one sense, cosmetic, since the preprocessor replaces embedded SQL by calls to library functions much like the functions in the standard SQL/CLI.

We shall give three examples of call-level interfaces. In this section, we cover the standard SQL/CLI, which is an adaptation of ODBC (Open Database Connectivity). We cover JDBC, which is a collection of classes that support database access from Java programs. Then, we explore PHP, which is a way to embed database access in Web pages described by HTML.

9.5.1 Introduction to SQL/CLI

A program written in C and using SQL/CLI (hereafter, just CLI) will include the header file `sqlcli.h`, from which it gets a large number of functions, type definitions, structures, and symbolic constants. The program is then able to create and deal with four kinds of records (structs, in C):

1. *Environments*. A record of this type is created by the application (client) program in preparation for one or more connections to the database server.
2. *Connections*. One of these records is created to connect the application program to the database. Each connection exists within some environment.
3. *Statements*. An application program can create one or more statement records. Each holds information about a single SQL statement, including an implied cursor if the statement is a query. At different times, the same CLI statement can represent different SQL statements. Every CLI statement exists within some connection.
4. *Descriptions*. These records hold information about either tuples or parameters. The application program or the database server, as appropriate, sets components of description records to indicate the names and types of attributes and/or their values. Each statement has several of these created implicitly, and the user can create more if needed. In our presentation of CLI, description records will generally be invisible.

Each of these records is represented in the application program by a *handle*, which is a pointer to the record. The header file `sqlcli.h` provides types for the handles of environments, connections, statements, and descriptions: `SQLHENV`, `SQLHDBC`, `SQLHSTMT`, and `SQLHDESC`, respectively, although we may think of them as pointers or integers. We shall use these types and also some other defined types with obvious interpretations, such as `SQL_CHAR` and `SQL_INTEGER`, that are provided in `sqlcli.h`.

We shall not go into detail about how descriptions are set and used. However, (handles for) the other three types of records are created by the use of a function

`SQLAllocHandle(hType, hIn, hOut)`

Here, the three arguments are:

1. *hType* is the type of handle desired. Use `SQL_HANDLE_ENV` for a new environment, `SQL_HANDLE_DBC` for a new connection, or `SQL_HANDLE_STMT` for a new statement.
2. *hIn* is the handle of the higher-level element in which the newly allocated element lives. This parameter is `SQL_NULL_HANDLE` if you want an environment; the latter name is a defined constant telling `SQLAllocHandle` that there is no relevant value here. If you want a connection handle, then *hIn* is the handle of the environment within which the connection will exist, and if you want a statement handle, then *hIn* is the handle of the connection within which the statement will exist.
3. *hOut* is the address of the handle that is created by `SQLAllocHandle`.

`SQLAllocHandle` also returns a value of type `SQLRETURN` (an integer). This value is 0 if no errors occurred, and there are certain nonzero values returned in the case of errors.

Example 9.18: Let us see how the function `worthRanges` of Fig. 9.8, which we used as an example of embedded SQL, would begin in CLI. Recall this function examines all the tuples of `MovieExec` and breaks their net worths into ranges. The initial steps are shown in Fig. 9.19.

```

1) #include sqlcli.h
2) SQLHENV myEnv;
3) SQLHDBC myCon;
4) SQLHSTMT execStat;
5) SQLRETURN errorCode1, errorCode2, errorCode3;

6) errorCode1 = SQLAllocHandle(SQL_HANDLE_ENV,
    SQL_NULL_HANDLE, &myEnv);
7) if(!errorCode1) {
8)     errorCode2 = SQLAllocHandle(SQL_HANDLE_DBC,
        myEnv, &myCon);
9)     if(!errorCode2)
10)        errorCode3 = SQLAllocHandle(SQL_HANDLE_STMT,
            myCon, &execStat); }
```

Figure 9.19: Declaring and creating an environment, a connection, and a statement

Lines (2) through (4) declare handles for an environment, connection, and statement, respectively; their names are `myEnv`, `myCon`, and `execStat`, respectively. We plan that `execStat` will represent the SQL statement

```
SELECT netWorth FROM MovieExec;
```

much as did the cursor `execCursor` in Fig. 9.8, but as yet there is no SQL statement associated with `execStat`. Line (5) declares three variables into which function calls can place their response and indicate an error. A value of 0 indicates no error occurred in the call.

Line (6) calls `SQLAllocHandle`, asking for an environment handle (the first argument), providing a null handle in the second argument (because none is needed when we are requesting an environment handle), and providing the address of `myEnv` as the third argument; the generated handle will be placed there. If line (6) is successful, lines (7) and (8) use the environment handle to get a connection handle in `myCon`. Assuming that call is also successful, lines (9) and (10) get a statement handle for `execStat`. $\square$

9.5.2 Processing Statements

At the end of Fig. 9.19, a statement record whose handle is `execStat`, has been created. However, there is as yet no SQL statement with which that record is associated. The process of associating and executing SQL statements with statement handles is analogous to the dynamic SQL described in Section 9.3.9. There, we associated the text of a SQL statement with what we called a “SQL variable,” using `PREPARE`, and then executed it using `EXECUTE`.

The situation in CLI is quite analogous, if we think of the “SQL variable” as a statement handle. There is a function

`SQLPrepare(sh, st, sl)`

that takes:

1. A statement handle *sh*,
2. A pointer to a SQL statement *st*, and
3. A length *sl* for the character string pointed to by *st*. If we don’t know the length, a defined constant `SQL_NTS` tells `SQLPrepare` to figure it out from the string itself. Presumably, the string is a “null-terminated string,” and it is sufficient for `SQLPrepare` to scan it until encountering the endmarker `'\0'`.

The effect of this function is to arrange that the statement referred to by the handle *sh* now represents the particular SQL statement *st*.

Another function

`SQLExecute(sh)`

causes the statement to which handle *sh* refers to be executed. For many forms of SQL statement, such as insertions or deletions, the effect of executing this statement on the database is obvious. Less obvious is what happens when the SQL statement referred to by *sh* is a query. As we shall see in Section 9.5.3,

there is an implicit cursor for this statement that is part of the statement record itself. The statement is in principle executed, so we can imagine that all the answer tuples are sitting somewhere, ready to be accessed. We can fetch tuples one at a time, using the implicit cursor, much as we did with real cursors in Sections 9.3 and 9.4.

Example 9.19: Let us continue with the function `worthRanges` that we began in Fig. 9.19. The following two function calls associate the query

```
SELECT netWorth FROM MovieExec;
```

with the statement referred to by handle `execStat`:

```
11) SQLPrepare(execStat, "SELECT netWorth FROM MovieExec",
    SQL_NTS);
12) SQLExecute(execStat);
```

These lines could appear right after line (10) of Fig. 9.19. Remember that `SQL_NTS` tells `SQLPrepare` to determine the length of the null-terminated string to which its second argument refers. $\square$

As with dynamic SQL, the prepare and execute steps can be combined into one if we use the function `SQLExecDirect`. An example that combines lines (11) and (12) above is:

```
SQLExecDirect(execStat, "SELECT netWorth FROM MovieExec",
    SQL_NTS);
```

9.5.3 Fetching Data From a Query Result

The function that corresponds to a `FETCH` command in embedded SQL or PSM is

```
SQLFetch(sh)
```

where *sh* is a statement handle. We presume the statement referred to by *sh* has been executed already, or the fetch will cause an error. `SQLFetch`, like all CLI functions, returns a value of type `SQLRETURN` that indicates either success or an error. The return value `SQL_NO_DATA` tells us tuples were left in the query result. As in our previous examples of fetching, this value will be used to get us out of a loop in which we repeatedly fetch new tuples from the result.

However, if we follow the `SQLExecute` of Example 9.19 by one or more `SQLFetch` calls, where does the tuple appear? The answer is that its components go into one of the description records associated with the statement whose handle appears in the `SQLFetch` call. We can extract the same component at each fetch by binding the component to a host-language variable, before we begin fetching. The function that does this job is:

`SQLBindCol(sh, colNo, colType, pVar, varSize, varInfo)`

The meanings of these six arguments are:

1. *sh* is the handle of the statement involved.
2. *colNo* is the number of the component (within the tuple) whose value we obtain.
3. *colType* is a code for the type of the variable into which the value of the component is to be placed. Examples of codes provided by `sqlcli.h` are `SQL_CHAR` for character arrays and strings, and `SQL_INTEGER` for integers.
4. *pVar* is a pointer to the variable into which the value is to be placed.
5. *varSize* is the length in bytes of the value of the variable pointed to by *pVar*.
6. *varInfo* is a pointer to an integer that can be used by `SQLBindCol` to provide additional information about the value produced.

Example 9.20: Let us redo the entire function `worthRanges` from Fig. 9.8, using CLI calls instead of embedded SQL. We begin as in Fig. 9.19, but for the sake of succinctness, we skip all error checking except for the test whether `SQLFetch` indicates that no more tuples are present. The code is shown in Fig. 9.20.

Line (3) declares the same local variables that the embedded-SQL version of the function uses, and lines (4) through (7) declare additional local variables using the types provided in `sqlcli.h`; these are variables that involve SQL in some way. Lines (4) through (6) are as in Fig. 9.19. New are the declarations on line (7) of `worth` (which corresponds to the shared variable of that name in Fig. 9.8) and `worthInfo`, which is required by `SQLBindCol`, but not used.

Lines (8) through (10) allocate the needed handles, as in Fig. 9.19, and lines (11) and (12) prepare and execute the SQL statement, as discussed in Example 9.19. In line (13), we see the binding of the first (and only) column of the result of this query to the variable `worth`. The first argument is the handle for the statement involved, and the second argument is the column involved, 1 in this case. The third argument is the type of the column, and the fourth argument is a pointer to the place where the value will be placed: the variable `worth`. The fifth argument is the size of that variable, and the final argument points to `worthInfo`, a place for `SQLBindCol` to put additional information (which we do not use here).

The balance of the function resembles closely lines (11) through (19) of Fig. 9.8. The while-loop begins at line (14) of Fig. 9.20. Notice that we fetch a tuple and check that we are not out of tuples, all within the condition of the while-loop, on line (14). If there is a tuple, then in lines (15) through (17) we determine the number of digits the integer (which is bound to `worth`) has and increment the appropriate count. After the loop finishes, i.e., all tuples returned

```

1) #include sqlcli.h
2) void worthRanges() {

3)     int i, digits, counts[15];
4)     SQLHENV myEnv;
5)     SQLHDBC myCon;
6)     SQLHSTMT execStat;
7)     SQLINTEGER worth, worthInfo;

8)     SQLAllocHandle(SQL_HANDLE_ENV,
9)         SQL_NULL_HANDLE, &myEnv);
10)    SQLAllocHandle(SQL_HANDLE_DBC, myEnv, &myCon);
11)    SQLAllocHandle(SQL_HANDLE_STMT, myCon, &execStat);
12)    SQLPrepare(execStat,
13)        "SELECT netWorth FROM MovieExec", SQL_NTS);
14)    SQLExecute(execStat);
15)    SQLBindCol(execStat, 1, SQL_INTEGER, &worth,
16)        sizeof(worth), &worthInfo);
17)    while(SQLFetch(execStat) != SQL_NO_DATA) {
18)        digits = 1;
19)        while((worth /= 10) > 0) digits++;
20)        if(digits <= 14) counts[digits]++;
21)    }
22)    for(i=0; i<15; i++)
23)        printf("digits = %d: number of execs = %d\n",
24)            i, counts[i]);
25) }

```

Figure 9.20: Grouping executive net worths: CLI version

by the statement execution of line (12) have been examined, the resulting counts are printed out at lines (18) and (19). □

9.5.4 Passing Parameters to Queries

Embedded SQL gives us the ability to execute a SQL statement, part of which consists of values determined by the current contents of shared variables. There is a similar capability in CLI, but it is rather more complicated. The steps needed are:

1. Use `SQLPrepare` to prepare a statement in which some portions, called *parameters*, are replaced by a question-mark. The *i*th question-mark represents the *i*th parameter.

Extracting Components with SQLGetData

An alternative to binding a program variable to an output of a query's result relation is to fetch tuples without any binding and then transfer components to program variables as needed. The function to use is `SQLGetData`, and it takes the same arguments as `SQLBindCol`. However, it only copies data once, and it must be used after each fetch in order to have the same effect as initially binding the column to a variable.

2. Use function `SQLBindParameter` to bind values to the places where the question-marks are found. This function has ten arguments, of which we shall explain only the essentials.
3. Execute the query with these bindings, by calling `SQLExecute`. Note that if we change the values of one or more parameters, we need to call `SQLExecute` again.

The following example will illustrate the process, as well as indicate the important arguments needed by `SQLBindParameter`.

Example 9.21: Let us reconsider the embedded SQL code of Fig. 9.6, where we obtained values for two variables `studioName` and `studioAddr` and used them as the components of a tuple, which we inserted into `Studio`. Figure 9.21 sketches how this process would work in CLI. It assumes that we have a statement handle `myStat` to use for the insertion statement.

```

/* get values for studioName and studioAddr */

1) SQLPrepare(myStat,
    "INSERT INTO Studio(name, address) VALUES(?, ?)",
    SQL_NTS);
2) SQLBindParameter(myStat, 1,..., studioName,...);
3) SQLBindParameter(myStat, 2,..., studioAddr,...);
4) SQLExecute(myStat);

```

Figure 9.21: Inserting a new studio by binding parameters to values

The code begins with steps (not shown) to give `studioName` and `studioAddr` values. Line (1) shows statement `myStat` being prepared to be an insertion statement with two parameters (the question-marks) in the `VALUE` clause. Then, lines (2) and (3) bind the first and second question-marks, to the current contents of `studioName` and `studioAddr`, respectively. Finally, line (4) executes the insertion. If the entire sequence of steps in Fig. 9.21, including the unseen work to obtain new values for `studioName` and `studioAddr`, are placed

in a loop, then each time around the loop, a new tuple, with a new name and address for a studio, is inserted into *Studio*. □

9.5.5 Exercises for Section 9.5

Exercise 9.5.1: Repeat the problems of Exercise 9.3.1, but write the code in C with CLI calls.

Exercise 9.5.2: Repeat the problems of Exercise 9.3.2, but write the code in C with CLI calls.

9.6 JDBC

Java Database Connectivity, or JDBC, is a facility similar to CLI for allowing Java programs to access SQL databases. The concepts resemble those of CLI, although Java's object-oriented flavor is evident in JDBC.

9.6.1 Introduction to JDBC

The first steps we must take to use JDBC are:

1. include the line:

```
import java.sql.*;
```

to make the JDBC classes available to your Java program.

2. Load a “driver” for the database system we shall use. The driver we need depends on which DBMS is available to us, but we load the needed driver with the statement:

```
Class.forName(<driver name>);
```

For example, to get the driver for a MySQL database, execute:

```
Class.forName("com.mysql.jdbc.Driver");
```

The effect is that a class called *DriverManager* is available. This class is analogous in many ways to the environment whose handle we get as the first step in using CLI.

3. Establish a connection to the database. A variable of class *Connection* is created if we apply the method *getConnection* to *DriverManager*.

The Java statement to establish a connection looks like:

```
Connection myCon = DriverManager.getConnection(<URL>,  
                                              <user name>, <password>);
```

That is, the method `getConnection` takes as arguments the URL for the database to which you wish to connect, your user name, and your password. It returns an object of class `Connection`, which we have chosen to call `myCon`.

Example 9.22: Each DBMS has its own way of specifying the URL in the `getConnection` method. For instance, if you want to connect to a MySQL database, the form of the URL is

```
jdbc:mysql://<host name>/<database name>
```

□

A JDBC Connection object is quite analogous to a CLI connection, and it serves the same purpose. By applying the appropriate methods to a `Connection` like `myCon`, we can create statement objects, place SQL statements “in” those objects, bind values to SQL statement parameters, execute the SQL statements, and examine results a tuple at a time.

9.6.2 Creating Statements in JDBC

There are two methods we can apply to a `Connection` object in order to create statements:

1. `createStatement()` returns a `Statement` object. This object has no associated SQL statement yet, so method `createStatement()` may be thought of as analogous to the CLI call to `SQLAllocHandle` that takes a connection handle and returns a statement handle.
2. `prepareStatement(Q)`, where Q is a SQL query passed as a string argument, returns a `PreparedStatement` object. Thus, we may draw an analogy between executing `prepareStatement(Q)` in JDBC with the two CLI steps in which we get a statement handle with `SQLAllocHandle` and then apply `SQLPrepare` to that handle and the query Q .

There are four different methods that execute SQL statements. Like the methods above, they differ in whether or not they take a SQL statement as an argument. However, these methods also distinguish between SQL statements that are queries and other statements, which are collectively called “updates.” Note that the SQL `UPDATE` statement is only one small example of what JDBC terms an “update.” The latter include all modification statements, such as inserts, and all schema-related statements such as `CREATE TABLE`. The four “execute” methods are:

- a) `executeQuery(Q)` takes a statement Q , which must be a query, and is applied to a `Statement` object. This method returns a `ResultSet` object, which is the set (bag, to be precise) of tuples produced by the query Q . We shall see how to access these tuples in Section 9.6.3.
- b) `executeQuery()` is applied to a `PreparedStatement` object. Since a prepared statement already has an associated query, there is no argument. This method also returns a `ResultSet` object.
- c) `executeUpdate(U)` takes a nonquery statement U and, when applied to a `Statement` object, executes U . The effect is felt on the database only; no `ResultSet` object is returned.
- d) `executeUpdate()`, with no argument, is applied to a `PreparedStatement` object. In that case, the SQL statement associated with the prepared statement is executed. This SQL statement must not be a query, of course.

Example 9.23: Suppose we have a `Connection` object `myCon`, and we wish to execute the query

```
SELECT netWorth FROM MovieExec;
```

One way to do so is to create a `Statement` object `execStat`, and then use it to execute the query directly.

```
Statement execStat = myCon.createStatement();
ResultSet worths = execStat.executeQuery(
    "SELECT netWorth FROM MovieExec");
```

The result of the query is a `ResultSet` object, which we have named `worths`. We'll see in Section 9.6.3 how to extract the tuples from `worths` and process them.

An alternative is to prepare the query immediately and later execute it. This approach would be preferable should we want to execute the same query repeatedly. Then, it makes sense to prepare it once and execute it many times, rather than having the DBMS prepare the same query many times. The JDBC steps needed to follow this approach are:

```
PreparedStatement execStat = myCon.prepareStatement(
    "SELECT netWorth FROM MovieExec");
ResultSet worths = execStat.executeQuery();
```

The result of executing the query is again a `ResultSet` object, which we have called `worths`. $\square$

Example 9.24: If we want to execute a parameterless nonquery, we can perform analogous steps in both styles. There is no result set, however. For instance, suppose we want to insert into `StarsIn` the fact that Denzel Washington starred in *Remember the Titans* in the year 2000. We may create and use a statement `starStat` in either of the following ways:

```
Statement starStat = myCon.createStatement();
starStat.executeUpdate("INSERT INTO StarsIn VALUES(" +
    "'Remember the Titans', 2000, 'Denzel Washington')");
```

or

```
PreparedStatement starStat = myCon.prepareStatement(
    "INSERT INTO StarsIn VALUES('Remember the Titans'," +
    "2000, 'Denzel Washington')");
starStat.executeUpdate();
```

Notice that each of these sequences of Java statements takes advantage of the fact that `+` is the Java operator that concatenates strings. Thus, we are able to extend SQL statements over several lines of Java, as needed. $\square$

9.6.3 Cursor Operations in JDBC

When we execute a query and obtain a result-set object, we may, in effect, run a cursor through the tuples of the result set. To do so, the `ResultSet` class provides the following useful methods:

1. `next()`, when applied to a `ResultSet` object, causes an implicit cursor to move to the next tuple (to the first tuple the first time it is applied). This method returns `FALSE` if there is no next tuple.
2. `getString(i)`, `getInt(i)`, `getFloat(i)`, and analogous methods for the other types that SQL values can take, each return the *i*th component of the tuple currently indicated by the cursor. The method appropriate to the type of the *i*th component must be used.

Example 9.25: Having obtained the result set `worths` as in Example 9.23, we may access its tuples one at a time. Recall that these tuples have only one component, of type integer. The form of the loop is:

```
while(worths.next()) {
    int worth = worths.getInt(1);
    /* process this net worth */
};
```

$\square$

9.6.4 Parameter Passing

As in CLI, we can use a question-mark in place of a portion of a query, and then bind values to those *parameters*. To do so in JDBC, we need to create a prepared statement, and we need to apply to that PreparedStatement object methods such as `setString(i, v)` or `setInt(i, v)` that bind the value *v*, which must be of the appropriate type for the method, to the *i*th parameter in the query.

Example 9.26: Let us mimic the CLI code in Example 9.21, where we prepared a statement to insert a new studio into relation `Studio`, with parameters for the name and address of that studio. The Java code to prepare this statement, set its parameters, and execute it is shown in Fig. 9.22. We continue to assume that connection object `myCon` is available to us.

```
1) PreparedStatement studioStat = myCon.prepareStatement(  
2)     "INSERT INTO Studio(name, address) VALUES(?, ?)";  
   /* get values for variables studioName and studioAddr  
      from the user */  
3) studioStat.setString(1, studioName);  
4) studioStat.setString(2, studioAddr);  
5) studioStat.executeUpdate();
```

Figure 9.22: Setting and using parameters in JDBC

In lines (1) and (2), we create and prepare the insertion statement. It has parameters for each of the values to be inserted. After line (2), we could begin a loop in which we repeatedly ask the user for a studio name and address, and place these strings in the variables `studioName` and `studioAddr`. This assignment is not shown, but represented by a comment. Lines (3) and (4) set the first and second parameters to the strings that are the current values of `studioName` and `studioAddr`, respectively. Finally, at line (5), we execute the insertion statement with the current values of its parameters. After line (5), we could go around the loop again, beginning with the steps represented by the comment. □

9.6.5 Exercises for Section 9.6

Exercise 9.6.1: Repeat Exercise 9.3.1, but write the code in Java using JDBC.

Exercise 9.6.2: Repeat Exercise 9.3.2, but write the code in Java using JDBC.

9.7 PHP

PHP is a scripting language for helping to create HTML Web pages. It provides support for database operations through an available library, much as JDBC

What Does PHP Stand For?

Originally, PHP was an acronym for “Personal Home Page.” More recently, it is said to be the recursive acronym “PHP: Hypertext Preprocessor” in the spirit of other recursive acronyms such as GNU (= “GNU is Not Unix”).

does. In this section we shall give a brief overview of PHP and show how database operations are performed in this language.

9.7.1 PHP Basics

All PHP code is intended to exist inside HTML text. A browser will recognize that text is PHP code by placing it inside a special tag, which looks like:

```
<?php
    PHP code goes here
?>
```

Many aspects of PHP, such as assignment statements, branches, and loops, will be familiar to the C or Java programmer, and we shall not cover them explicitly. However, there are some interesting features of PHP of which we should be aware.

Variables

Variables are untyped and need not be declared. All variable names begin with \$.

Often, a variable will be declared to be a member of a “class,” in which case certain *functions* (analogous to methods in Java) may be applied to that variable. The function-application operator is `->`, comparable to the dot in Java or C++.

Strings

String values in PHP can be surrounded by either single or double quotes, but there is an important difference. Strings surrounded by single quotes are treated literally, just like SQL strings. However, when a string has double quotes around it, any variable names within the string are replaced by their values.

Example 9.27: In the following code:

```
$foo = 'bar';
$x = 'Step up to the $foo';
```

the value of `$x` is `Step up to the $foo`. However, if the following code is executed instead:

```
$foo = "bar";  
$x = "Step up to the $foo";
```

the value of `$x` is `Step up to the bar`. It doesn't matter whether `bar` has single or double quotes, since it contains no dollar-signs and therefore no variables. However, the variable `$foo` is replaced only when surrounded by double quotes, as in the second example. $\square$

Concatenation of strings is denoted by a dot. Thus,

```
$y = "$foo" . 'bar';
```

gives `$y` the value `barbar`.

9.7.2 Arrays

PHP has ordinary arrays (called *numeric*), which are indexed `0, 1, \dots`. It also has arrays that are really mappings, called *associative arrays*. The indexes (*keys*) of an associative array can be any strings, and the array associates a single value with each key. Both kinds of arrays use the conventional square brackets for indexing, but for associative arrays, an array element is represented by:

`<key> => <value>`

Example 9.28: The following line:

```
$a = array(30,20,10,0);
```

sets `$a` to be a numeric array of length four, with `$a[0]` equal to 30, `$a[1]` equal to 20, and so on. $\square$

Example 9.29: The following line:

```
$seasons = array('spring' => 'warm', 'summer' => 'hot',  
                'fall' => 'warm', 'winter' => 'cold');
```

makes `$seasons` be an array of length four, but it is an associative array. For instance, `$seasons['summer']` has the value `'hot'`. $\square$

9.7.3 The PEAR DB Library

PHP has a collection of libraries called PEAR (PHP Extension and Application Repository). One of these libraries, DB, has generic functions that are analogous to the methods of JDBC. We tell the function `DB::connect` which vendor's DBMS we wish to access, but none of the other functions of DB need to know about which DBMS we are using. Note that the double colon in `DB::connect` is PHP's way of saying "the function `connect` in the DB library." We make the DB library available to our PHP program with the statement:

```
include(DB.php);
```

9.7.4 Creating a Database Connection Using DB

The form of an invocation of the `connect` function is:

```
$myCon = DB::connect(<vendor>://<user name>:<password>
                    <host name>/<database name>);
```

The components of this call are like those in the analogous JDBC statement that creates a connection (see Section 9.6.1). The one exception is the vendor, which is a code used by the DB library. For example, `mysql` is the code for recent versions of the MySQL database.

After executing this statement, the variable `$myCon` is a connection. Like all PHP variables, `$myCon` can change its type. But as long as it is a connection, we may apply to it a number of useful functions that enable us to manipulate the database to which the connection was made. For example, we can disconnect from the database by

```
$myCon->disconnect();
```

Remember that `->` is the PHP way of applying a function to an "object."

9.7.5 Executing SQL Statements

All SQL statements are referred to as "queries" and are executed by the function `query`, which takes the statement as an argument and is applied to the connection variable.

Example 9.30: Let us duplicate the insertion statement of Example 9.24, where we inserted Denzel Washington and *Remember the Titans* into the `Stars-In` table. Assuming that `$myCon` has connected to our movie database, We can simply say:

```
$result = $myCon->query("INSERT INTO StarsIn VALUES(" .
                        "'Denzel Washington', 2000, 'Remember the Titans')");
```

Note that the dot concatenates the two strings that form the query. We only broke the query into two strings because it was necessary to break it over two lines.

The variable `$result` will hold an error code if the insert-statement failed to execute. If the “query” were really a SQL query, then `$result` is a cursor to the tuples of the result (see Section 9.7.6). $\square$

PHP allows SQL to have parameters, denoted by question-marks, as we shall discuss in Section 9.7.7. However, the ability to expand variables in doubly quoted strings gives us another easy way to execute SQL statements that depend on user input. In particular, since PHP is used within Web pages, there are built-in ways to exploit HTML’s capabilities.

We often get information from a user of a Web page by showing them a form and having their answers “posted.” PHP provides an associative array called `$_POST` with all the information provided by the user. Its keys are the names of the form elements, and the associated values are what the user has entered into the form.

Example 9.31: Suppose we ask the user to fill out a form whose elements are `title`, `year`, and `starName`. These three values will form a tuple that we may insert into the table `StarsIn`. The statement:

```
$result = $myCon->query("INSERT INTO StarsIn VALUES(
    $_POST['title'], $_POST['year'], $_POST['starName']");
```

will obtain the posted values for these three form elements. Since the query argument is a double-quoted string, PHP evaluates terms like `$_POST['title']` and replaces them by their values. $\square$

9.7.6 Cursor Operations in PHP

When the `query` function gets a true query as argument, it returns a result object, that is, a list of tuples. Each tuple is a numeric array, indexed by integers starting at 0. The essential function that we can apply to a result object is `fetchRow()`, which returns the next row, or 0 (false) if there is no next row.

```
1) $worths = $myCon->query("SELECT netWorth FROM MovieExec");
2) while ($tuple = $worths->fetchRow()) {
3)     $worth = $tuple[0];
        // process this value of $worth
}
```

Figure 9.23: Finding and processing net worths in PHP

Example 9.32: In Fig. 9.23 is PHP code that is the equivalent of the JDBC in Examples 9.23 and 9.25. It assumes that connection `$myCon` is available, as before.

Line (1) passes the query to the connection `$myCon`, and the result object is assigned to the variable `$worths`. We then enter a loop, in which we repeatedly get a tuple from the result and assign this tuple to the variable `$tuple`, which technically becomes an array of length 1, with only a component for the column `netWorth`. As in C, the value returned by `fetchRow()` becomes the value of the condition in the while-statement. Thus, if no tuple is found, this value, 0, terminates the loop. At line (3), the value of the tuple's first (and only) component is extracted and assigned to the variable `$worth`. We do not show the processing of this value. $\square$

9.7.7 Dynamic SQL in PHP

As in JDBC, PHP allows a SQL query to contain question-marks. These question-marks are placeholders for values that can be filled in later, during the execution of the statement. The process of doing so is as follows.

We may apply `prepare` and `execute` functions to a connection; these functions are analogous to similarly named functions discussed in Section 9.3.9 and elsewhere. Function `prepare` takes a SQL statement as argument and returns a prepared version of that statement. Function `execute` takes two arguments: the prepared statement and an array of values to substitute for the question-marks in the statement. If there is only one question-mark, a simple variable, rather than an array, suffices.

Example 9.33: Let us again look at the problem of Example 9.26, where we prepared to insert many name-address pairs into relation `Studio`. To begin, we prepare the query, with parameters, by:

```
$prepQuery = $myCon->prepare("INSERT INTO Studio(name, " .  
                             "address) VALUES(?,?)");
```

Now, `$prepQuery` is a “prepared query.” We can use it as an argument to `execute` along with an array of two values, a studio name and address. For example, we could perform the following statements:

```
$args = array('MGM', 'Los Angeles');  
$result = $myCon->execute($prepQuery, $args);
```

The advantage of this arrangement is the same as for all implementations of dynamic SQL. If we insert many different tuples this way, we only have to prepare the insertion statement once and can execute it many times. $\square$

9.7.8 Exercises for Section 9.7

Exercise 9.7.1: Repeat Exercise 9.3.1, but write the code using PHP.

Exercise 9.7.2: Repeat Exercise 9.3.2, but write the code using PHP.

! Exercise 9.7.3: In Example 9.31 we exploited the feature of PHP that strings in double-quotes have variables expanded. How essential is this feature? Could we have done something analogous in JDBC? If so, how?

9.8 Summary of Chapter 9

- ◆ *Three-Tier Architectures:* Large database installations that support large-scale user interactions over the Web commonly use three tiers of processes: web servers, application servers, and database servers. There can be many processes active at each tier, and these processes can be at one processor or distributed over many processors.
- ◆ *Client-Server Systems in the SQL Standard:* The standard talks of SQL clients connecting to SQL servers, creating a connection (link between the two processes) and a session (sequence of operations). The code executed during the session comes from a module, and the execution of the module is called a SQL agent.
- ◆ *The Database Environment:* An installation using a SQL DBMS creates a SQL environment. Within the environment, database elements such as relations are grouped into (database) schemas, catalogs, and clusters. A catalog is a collection of schemas, and a cluster is the largest collection of elements that one user may see.
- ◆ *Impedance Mismatch:* The data model of SQL is quite different from the data models of conventional host languages. Thus, information passes between SQL and the host language through shared variables that can represent components of tuples in the SQL portion of the program.
- ◆ *Embedded SQL:* Instead of using a generic query interface to express SQL queries and modifications, it is often more effective to write programs that embed SQL queries in a conventional host language. A preprocessor converts the embedded SQL statements into suitable function calls of the host language.
- ◆ *Cursors:* A cursor is a SQL variable that indicates one of the tuples of a relation. Connection between the host language and SQL is facilitated by having the cursor range over each tuple of the relation, while the components of the current tuple are retrieved into shared variables and processed using the host language.

- ◆ *Dynamic SQL*: Instead of embedding particular SQL statements in a host-language program, the host program may create character strings that are interpreted by the SQL system as SQL statements and executed.
- ◆ *Persistent Stored Modules*: We may create collections of procedures and functions as part of a database schema. These are written in a special language that has all the familiar control primitives, as well as SQL statements.
- *The Call-Level Interface*: There is a standard library of functions, called SQL/CLI or ODBC, that can be linked into any C program. These functions give capabilities similar to embedded SQL, but without the need for a preprocessor.
- ◆ *JDBC*: Java Database Connectivity is a collection of Java classes analogous to CLI for connecting Java programs to a database.
- ◆ *PHP*: Another popular system for implementing a call-level interface is PHP. This language is found embedded in HTML pages and enables these pages to interact with a database.

9.9 References for Chapter 9

The PSM standard is [4], and [5] is a comprehensive book on the subject. Oracle's version of PSM is called PL/SQL; a summary can be found in [2]. SQL Server has a version called Transact-SQL [6]. IBM's version is SQL PL [1].

[3] is a popular reference on JDBC. [7] is one on PHP, which was originally developed by one of the book's authors, R. Lerdorf.

1. D. Bradstock et al., *DB2 SQL Procedure Language for Linux, Unix, and Windows*, IBM Press, 2005.
2. Y.-M. Chang et al., "Using Oracle PL/SQL"
<http://infolab.stanford.edu/~ullman/fcdb/oracle/or-plsql.html>
3. M. Fisher, J. Ellis, and J. Bruce, *JDBC API Tutorial and Reference*, Prentice-Hall, Upper Saddle River, NJ, 2003.
4. ISO/IEC Report 9075-4, 2003.
5. J. Melton, *Understanding SQL's Stored Procedures: A Complete Guide to SQL/PSM*, Morgan-Kaufmann, San Francisco, 1998.
6. Microsoft Corp., "Transact-SQL Reference"
<http://msdn2.microsoft.com/en-us/library/ms189826.aspx>
7. K. Tatroe, R. Lerdorf, and P. MacIntyre, *Programming PHP*, O'Reilly Media, Cambridge, MA, 2006.

